Homework 10

Eric Chen

All code is in Python and uses the `rlt` package (`RLT_reg` and `RLT_cla`) with reinforcement turned off (`reinforcement=False`). Node size is `min_samples_leaf` and the number of candidate predictors is `mtry`. The random seed for both questions is **432**.

```
import warnings; warnings.filterwarnings("ignore")
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from rlt import RLT_reg, RLT_cla

SEED = 432
BLUE, ORANGE, GRAY = "#0072B2", "#D55E00", "0.45"
```

Question 1: Bias-variance trade-off simulation

Five predictors are jointly normal with mean 0, variance 1, and pairwise correlation 0.3, and $Y = X_1 + X_3 + X_5 + \varepsilon$ with $\varepsilon \sim N(0,1)$. (a) Run a 100-repetition simulation of 500-tree forests, varying node size with `mtry` = 3 and varying `mtry` with node size 10, and plot the averaged squared bias and variance. (b) Discuss the results.

Solution

The 200 test points $\mathbf{x}_1, \ldots, \mathbf{x}_{200}$ are drawn once. In repetition $r = 1, \ldots, 100$ a new training set of 200 observations is drawn, and every tuning setting is fit to that same training set with the same tree seed. Each tree draws 200 observations with replacement (`resample_replace=True`, `resample_prob=1.0`). Let $\hat{f}_r(\mathbf{x}_j)$ be the prediction at test point j in repetition r and $\bar{f}(\mathbf{x}_j) = \frac{1}{100}\sum_r \hat{f}_r(\mathbf{x}_j)$. For each setting,

$$\text{Bias}^2 = \frac{1}{200}\sum_{j=1}^{200}\big(\bar{f}(\mathbf{x}_j) - f(\mathbf{x}_j)\big)^2, \qquad \text{Var} = \frac{1}{200}\sum_{j=1}^{200}\frac{1}{100}\sum_{r=1}^{100}\big(\hat{f}_r(\mathbf{x}_j) - \bar{f}(\mathbf{x}_j)\big)^2,$$

so that $\text{Bias}^2 + \text{Var}$ is the average mean squared error for estimating f. The expected test error for a new Y adds the noise variance 1 to this.

The same loop also fits the 100-observation version needed for part (b) (`resample_prob=0.5`, node sizes $1, \ldots, 30$, `mtry` = 3). The setting `mtry` = 3 with node size 10 appears in both comparisons and is fit once.

```
rng = np.random.default_rng(SEED)
p, rho, n_train, n_test, n_rep = 5, 0.3, 200, 200, 100
Sigma = np.full((p, p), rho) + (1 - rho) * np.eye(p)
def f(X):
    return X[:, 0] + X[:, 2] + X[:, 4]

X_test = rng.multivariate_normal(np.zeros(p), Sigma, size=n_test)
f_test = f(X_test)

nodesizes, mtrys = [1, 5, 10, 20, 30], [1, 2, 3, 4, 5]
# (mtry, node size, resample_prob); resample_prob 0.5 draws 100 of 200 observations
settings = sorted({(3, k, 1.0) for k in nodesizes} | {(m, 10, 1.0) for m in mtrys}
                  | {(3, k, 0.5) for k in nodesizes})
pred = {s: np.empty((n_rep, n_test)) for s in settings}
```

```
for r in range(n_rep):
    X = rng.multivariate_normal(np.zeros(p), Sigma, size=n_train)
    y = f(X) + rng.normal(size=n_train)
    tree_seed = int(rng.integers(2**31 - 1))
    for m, k, rp in settings:
        fit = RLT_reg(n_estimators=500, mtry=m, min_samples_leaf=k,
                      resample_replace=True, resample_prob=rp,
                      reinforcement=False, n_jobs=1, random_state=tree_seed)
        pred[(m, k, rp)][r] = fit.fit(X, y).predict(X_test)

rows = []
for (m, k, rp), P in pred.items():
    fbar = P.mean(axis=0)
    rows.append(dict(mtry=m, nodesize=k, resample_prob=rp,
                     bias2=np.mean((fbar - f_test) ** 2), var=np.mean(P.var(axis=0))))
q1 = pd.DataFrame(rows)
q1["mse"] = q1["bias2"] + q1["var"]
```

(a)

```
def bv_plot(ax, d, x, xlabel, title):
    ax.plot(d[x], d["bias2"], "o-", color=BLUE, label="squared bias")
    ax.plot(d[x], d["var"], "s-", color=ORANGE, label="variance")
    ax.plot(d[x], d["mse"], "^--", color=GRAY, lw=1, label="bias² + variance")
    ax.set(xticks=d[x], xlabel=xlabel, ylabel="average over 200 test points",
           title=title, ylim=(0, None))
    ax.legend(fontsize=8, frameon=False)
```

```
d_node = q1[(q1.mtry == 3) & (q1.resample_prob == 1.0)].sort_values("nodesize")
fig, ax = plt.subplots(figsize=(6, 3.8))
bv_plot(ax, d_node, "nodesize", "node size (min_samples_leaf)", "mtry = 3")
fig.tight_layout(); plt.show()
```

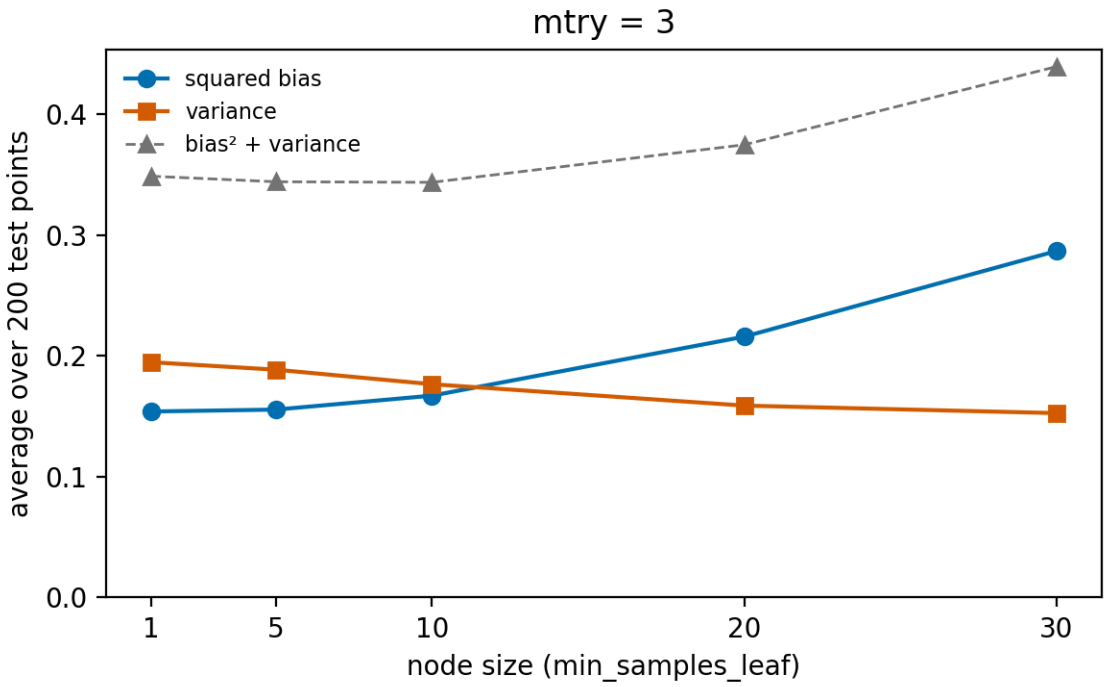


Figure 1: Averaged squared bias and variance against node size, with mtry = 3 and 200 observations drawn with replacement per tree (100 repetitions).

```
d_mtry = q1[(q1.nodesize == 10) & (q1.resample_prob == 1.0)].sort_values("mtry")
fig, ax = plt.subplots(figsize=(6, 3.8))
bv_plot(ax, d_mtry, "mtry", "mtry", "node size = 10")
fig.tight_layout(); plt.show()
```

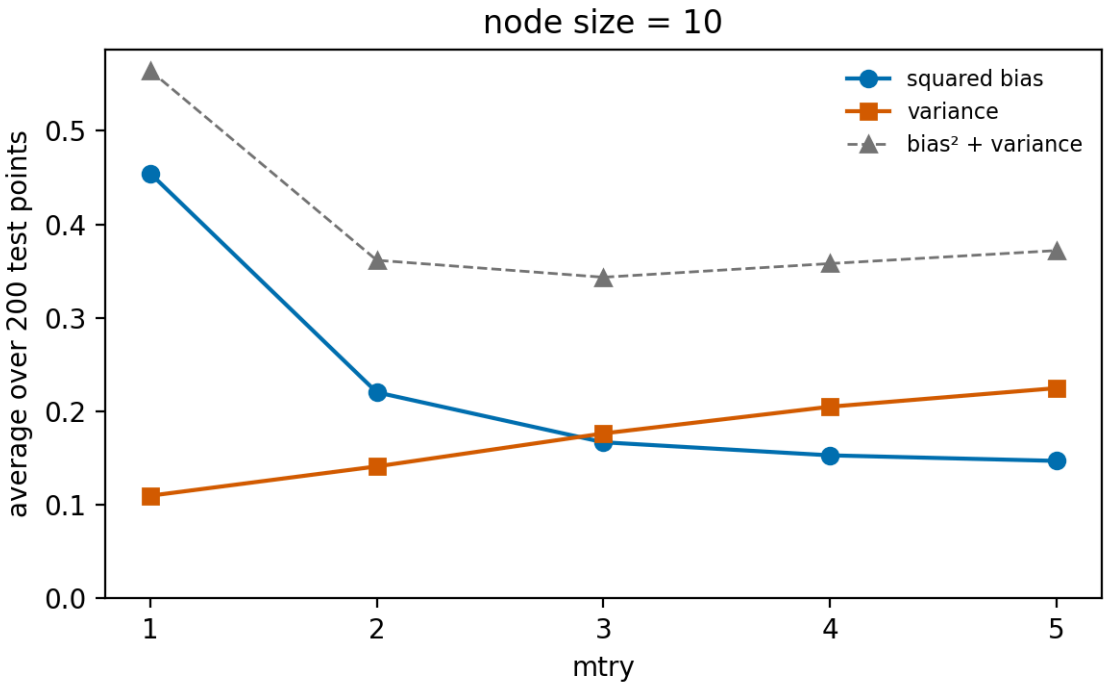


Figure 2: Averaged squared bias and variance against mtry, with node size 10 and 200 observations drawn with replacement per tree (100 repetitions).

Table 1: Averaged squared bias, variance, and their sum for each setting (200 observations per tree).

```
show = pd.concat([d_node.assign(vary="node size"), d_mtry.assign(vary="mtry")])
print(show[["vary", "mtry", "nodesize", "bias2", "var", "mse"]]
      .to_string(index=False, float_format=lambda v: f"{v:.3f}"))
```

```
     vary  mtry  nodesize  bias2   var   mse
node size     3         1  0.154 0.195 0.349
node size     3         5  0.155 0.188 0.344
node size     3        10  0.167 0.176 0.343
node size     3        20  0.216 0.159 0.375
node size     3        30  0.287 0.153 0.439
     mtry     1        10  0.455 0.110 0.565
     mtry     2        10  0.220 0.141 0.362
     mtry     3        10  0.167 0.176 0.343
     mtry     4        10  0.153 0.205 0.358
     mtry     5        10  0.147 0.225 0.372
```

(b)

- **Large node size.** As node size grows from 1 to 30, squared bias rises steadily (0.154 to 0.287) and variance falls (0.195 to 0.153). Large terminal nodes average the responses over a wide region of $\mathbf{x}$, so each tree is a coarser step function that flattens the linear trend $x_1 + x_3 + x_5$, especially at test points in the tails. Averaging over more observations makes each tree's leaf means less noisy, which lowers variance. At node size 30 the extra bias outweighs the variance saving, and the sum (0.439) is the worst of the five. The sum is nearly flat for node sizes 1 to 10 (about 0.344 to 0.349), where the bias and variance changes roughly cancel. So node size 30 underfits in this example.
- **Large `mtry`.** With node size 10, increasing `mtry` from 1 to 5 lowers squared bias (0.455 to 0.147) but raises variance (0.110 to 0.225). The benefit is that each split is more likely to use one of the three signal variables X_1, X_3, X_5 rather than the noise variables X_2, X_4. Each tree then fits f better, which matters a lot here because 2 of the 5 predictors carry no signal. The drawback is that the trees become more similar to each other. With `mtry` = 5 every tree searches all variables and the forest is just bagging, so averaging over trees removes less of the variance. The sum is smallest at `mtry` = 3 (0.344) and larger at both ends: `mtry` = 1 is dominated by bias and `mtry` = 5 by variance.

```
fig, ax = plt.subplots(figsize=(6, 3.6))
for rp, col, mk, lab in [(1.0, BLUE, "o", "200 per tree"), (0.5, ORANGE, "s", "100 per tree")]:
    d = q1[(q1.mtry == 3) & (q1.resample_prob == rp)].sort_values("nodesize")
    ax.plot(d["nodesize"], d["mse"], marker=mk, color=col, label=lab)
ax.set(xticks=nodesizes, xlabel="node size (min_samples_leaf)", ylabel="bias² + variance")
ax.legend(fontsize=8, frameon=False)
fig.tight_layout(); plt.show()
print(q1[q1.mtry == 3].pivot(index="nodesize", columns="resample_prob",
      values=["bias2", "var", "mse"]).round(3).to_string())
```

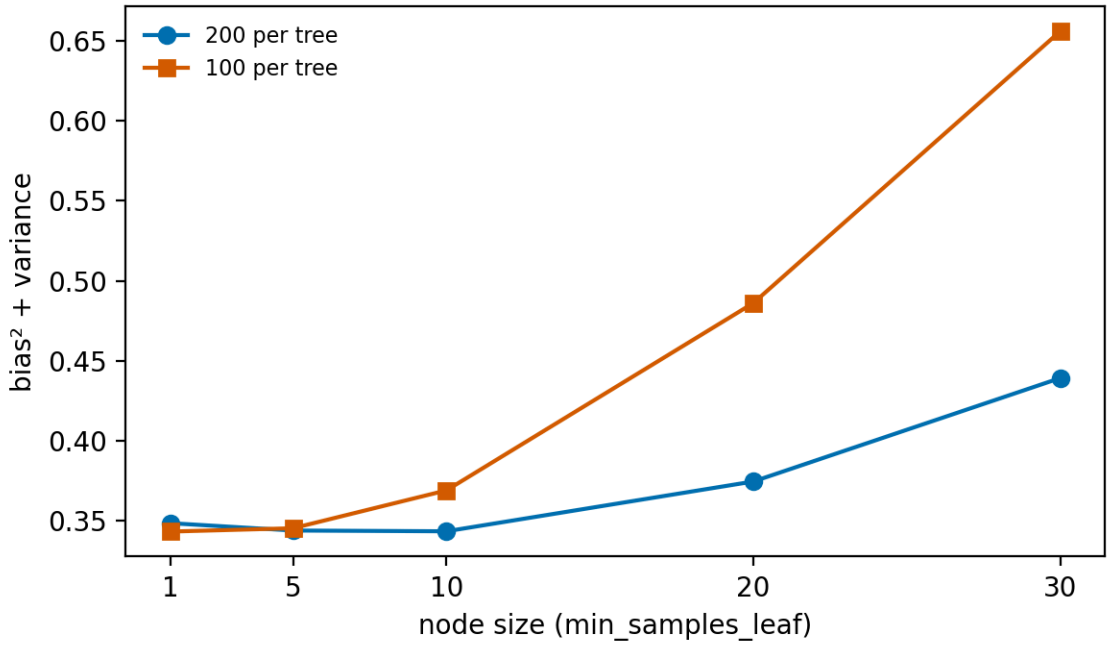


Figure 3: Bias² + variance against node size for 200 versus 100 observations drawn with replacement per tree (mtry = 3, same training data and test points).

	bias2		var		mse	
resample_prob	0.5	1.0	0.5	1.0	0.5	1.0
nodesize						
1	0.212	0.154	0.131	0.195	0.343	0.349
5	0.219	0.155	0.127	0.188	0.345	0.344
10	0.251	0.167	0.118	0.176	0.369	0.343
20	0.373	0.216	0.113	0.159	0.486	0.375
30	0.540	0.287	0.116	0.153	0.656	0.439

- **100 observations per tree.** I would expect the preferred node size to shift toward **smaller** values. Drawing 100 instead of 200 observations makes each tree less correlated with the others, so averaging over 500 trees already removes much of the variance (variance at node size 1 drops from 0.195 to 0.131). But each tree now has only about half as many distinct observations, so a leaf of a given size covers a region of $\mathbf{x}$ about twice as wide and the bias at a given node size is larger. Since subsampling already controls variance, the forest gains less from large leaves and loses more bias from them, so smaller leaves are favored. The extra simulation agrees: with 100 observations the sum is smallest at node size 1 (0.343) and rises quickly after 5 (0.656 at node size 30). With 200 observations the minimum is spread over node sizes 1 to 10 and the curve rises more slowly. The difference between node sizes 1 and 5 is small in both cases, so the exact minimizer is not a reliable pattern.

Question 2: Tuning random forests for MNIST

Use the 2,000-observation MNIST file. Keep digits 1, 6, and 7, with the first 1,000 rows as training data and the last 1,000 as test data, and use the first 20 principal components of the training pixels (centered, not scaled) as predictors. (a) Report the OOB error of a default 1,000-tree forest. (b) Tune `mtry` and node size by OOB error and report the OOB permutation importance of the selected forest. (c) Report the test misclassification rate and confusion matrix.

Solution

`PCA` in scikit-learn centers each pixel at its training mean and does not scale it. The test observations are projected with `pca.transform`, which subtracts the same training means and uses the same loadings. Each tree uses a bootstrap sample of the training-set size (`resample_replace=True`, `resample_prob=1.0`). All forests use `random_state=432`.

```
mnist = pd.read_csv("data/mnist-first2000.csv.gz")
digits = [1, 6, 7]
train, test = mnist.iloc[:1000], mnist.iloc[1000:]
train, test = train[train.Digit.isin(digits)], test[test.Digit.isin(digits)]
y_train, y_test = train.Digit.to_numpy(), test.Digit.to_numpy()

# Exact SVD; the default solver here is randomized and changes between runs
pca = PCA(n_components=20, svd_solver="full").fit(train.drop(columns="Digit"))
pcs = [f"PC{j}" for j in range(1, 21)]
Z_train = pd.DataFrame(pca.transform(train.drop(columns="Digit")), columns=pcs)
Z_test = pd.DataFrame(pca.transform(test.drop(columns="Digit")), columns=pcs)

print(f"training n = {len(train)}, test n = {len(test)}")
print("training counts:", train.Digit.value_counts().sort_index().to_dict())
print("test counts:    ", test.Digit.value_counts().sort_index().to_dict())
print(f"variance explained by 20 PCs: {pca.explained_variance_ratio_.sum():.3f}")
```

```
training n = 327, test n = 317
training counts: {1: 116, 6: 94, 7: 117}
test counts:     {1: 104, 6: 106, 7: 107}
variance explained by 20 PCs: 0.771
```

(a)

```
base = dict(n_estimators=1000, resample_replace=True, resample_prob=1.0,
            reinforcement=False, n_jobs=1, random_state=SEED)
fit0 = RLT_cla(**base).fit(Z_train, y_train)
n_wrong = round(fit0.oob_error_ * len(y_train))
print(f"default mtry = {fit0.params_.mtry}, default node size = {fit0.params_.nmin}")
print(f"OOB misclassification rate = {fit0.oob_error_:.4f} ({n_wrong} of {len(y_train)})")
```

```
default mtry = 10, default node size = 5
OOB misclassification rate = 0.0398 (13 of 327)
```

The default forest uses $\texttt{mtry} = \lfloor 20/2 \rfloor = 10$ and node size 5. Its OOB misclassification rate is reported above.

(b) The grid crosses $\texttt{mtry} \in \{1, 2, 3, 4, 5, 6, 8, 10, 15, 20\}$ with node size $\in \{1, 2, 3, 5, 10, 15, 20, 30\}$, for 80 settings, all with the same seed. The selected setting has the smallest OOB misclassification rate. With 327 training observations the OOB error moves in steps of $1/327 \approx 0.003$, so ties are likely. Ties are broken by the OOB Brier score $\frac{1}{n}\sum_i \sum_c (\hat{p}_{ic} - 1\{y_i = c\})^2$, which uses the same OOB class probabilities and so still looks only at training data.

```
mtry_grid = [1, 2, 3, 4, 5, 6, 8, 10, 15, 20]
node_grid = [1, 2, 3, 5, 10, 15, 20, 30]
onehot = (y_train[:, None] == np.array(digits)).astype(float)

rows = []
for m in mtry_grid:
    for k in node_grid:
        fit = RLT_cla(mtry=m, min_samples_leaf=k, **base).fit(Z_train, y_train)
        rows.append(dict(mtry=m, nodesize=k, oob=fit.oob_error_,
                         brier=np.mean(np.sum((fit.oob_prob_ - onehot) ** 2, axis=1))))
grid = pd.DataFrame(rows)
best = grid.sort_values(["oob", "brier"]).iloc[0]
m_best, k_best = int(best.mtry), int(best.nodesize)
```

```
print(f"selected: mtry = {m_best}, node size = {k_best}, "
      f"OOB error = {best.oob:.4f}, OOB Brier = {best.brier:.4f}")
```

```
selected: mtry = 1, node size = 1, OOB error = 0.0214, OOB Brier = 0.1430
```

Table 2: OOB misclassification rate for each setting (rows: node size, columns: mtry).

```
print(grid.pivot(index="nodesize", columns="mtry", values="oob")
      .to_string(float_format=lambda v: f"{v:.3f}"))
```

```
mtry        1     2     3     4     5     6     8     10    15    20
nodesize
1        0.021 0.034 0.034 0.037 0.037 0.037 0.040 0.046 0.052 0.049
2        0.021 0.031 0.034 0.037 0.037 0.040 0.040 0.046 0.052 0.049
3        0.024 0.037 0.037 0.037 0.037 0.040 0.040 0.040 0.052 0.049
5        0.021 0.037 0.037 0.037 0.037 0.037 0.040 0.040 0.052 0.049
10       0.021 0.034 0.040 0.037 0.037 0.037 0.043 0.043 0.052 0.049
15       0.021 0.034 0.040 0.037 0.037 0.037 0.040 0.043 0.052 0.049
20       0.024 0.037 0.040 0.037 0.037 0.037 0.040 0.043 0.052 0.049
30       0.031 0.037 0.040 0.037 0.037 0.040 0.040 0.043 0.052 0.049
```

```
fit_best = RLT_cla(mtry=m_best, min_samples_leaf=k_best, importance="permute",
                   **base).fit(Z_train, y_train)
vi = pd.Series(fit_best.feature_importances_, index=pcs)
fig, ax = plt.subplots(figsize=(7, 3.4))
ax.bar(pcs, vi, color=BLUE)
ax.axhline(0, color="0.3", lw=0.8)
ax.set(ylabel="permutation importance",
       title=f"mtry = {m_best}, node size = {k_best}")
ax.tick_params(axis="x", labelrotation=60, labelsize=8)
fig.tight_layout(); plt.show()
print(vi.sort_values(ascending=False).round(4).to_string())
```

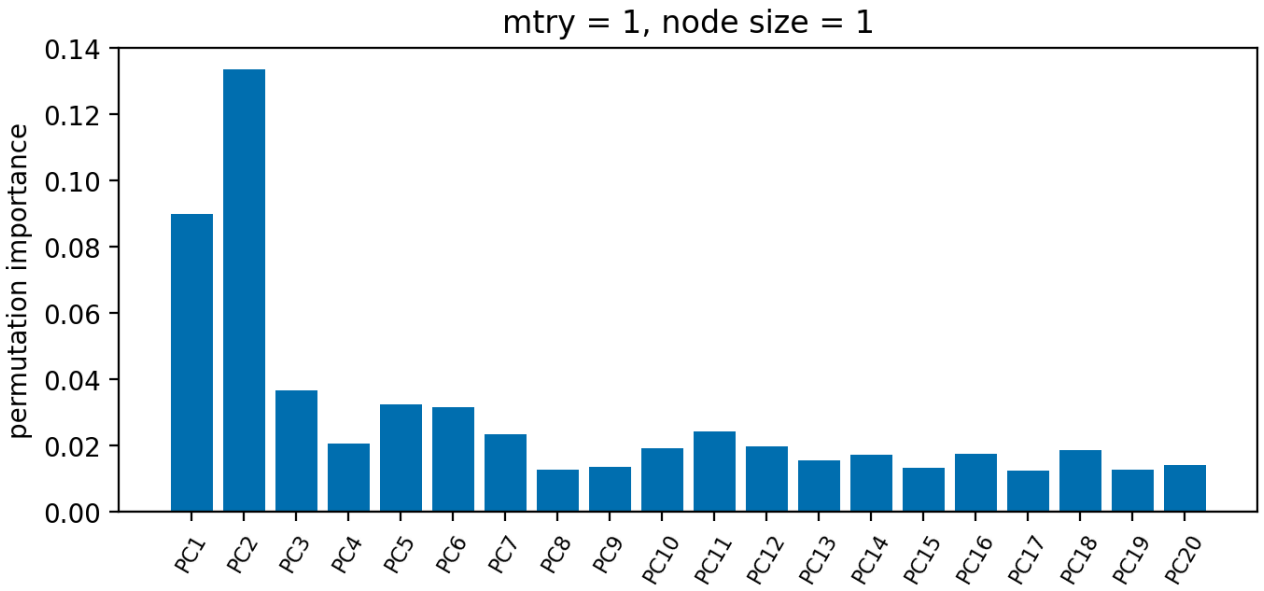


Figure 4: OOB permutation variable importance of the 20 principal components in the selected forest.

```
PC2     0.1335
PC1     0.0899
PC3     0.0367
PC5     0.0324
PC6     0.0315
PC11    0.0241
PC7     0.0234
PC4     0.0205
PC12    0.0196
PC10    0.0191
PC18    0.0184
PC16    0.0175
PC14    0.0170
PC13    0.0153
PC20    0.0140
PC9     0.0135
PC15    0.0131
PC19    0.0126
PC8     0.0126
PC17    0.0124
```

```python
print(f"OOB error of the refit forest: {fit_best.oob_error_:.4f}")
print("OOB error by mtry (minimum over node sizes):")
print(grid.groupby("mtry").oob.min().round(4).to_string())
print(f"top 3 PCs carry {vi.nlargest(3).sum() / vi.clip(lower=0).sum():.0%} of the positive
importance")
```

```
OOB error of the refit forest: 0.0214
OOB error by mtry (minimum over node sizes):
mtry
1     0.0214
2     0.0306
3     0.0336
4     0.0367
5     0.0367
6     0.0367
8     0.0398
10    0.0398
15    0.0520
20    0.0489
top 3 PCs carry 45% of the positive importance
```

Comments.

- **Selected values: `mtry` = 1 and node size 1**, with OOB error 0.021 (7 of 327). Node sizes 2, 5, 10, and 15 tie with it at `mtry` = 1, and the Brier score breaks the tie in favor of node size 1. The default forest (`mtry` = 10, node size 5) has OOB error 0.040 (13 of 327), so tuning nearly halves the OOB errors. Even so, the difference is only 6 observations, and settings near the top of the grid differ by one or two observations, so the exact ranking among them is not very precise.
- **`mtry` matters much more than node size.** The best OOB error for each `mtry` rises from 0.021 at `mtry` = 1 to 0.031 at 2, about 0.037 for 4 to 6, 0.040 at 8 and 10, and about 0.05 at 15 and 20. Within a column the error usually changes by at most one or two observations. It only clearly grows at node size 30 when `mtry` is 1. Small `mtry` working best is reasonable here. The principal components are uncorrelated by construction, and many of them carry some information about the digit. Choosing among one random component per split makes the trees use many different

components and keeps them less correlated, so averaging 1,000 of them removes more variance. With deep trees, the extra bias from weaker individual splits is small.

- **Importance.** PC2 (0.134) and PC1 (0.090) are clearly the most important, followed by PC3, PC5, and PC6 (about 0.03 each). The top three components carry 45% of the total importance. The remaining components all have small positive importance between about 0.012 and 0.024, and none is near zero or negative. The leading components describe the largest pixel-level variation in the training digits, so they likely capture the main shape differences between 1, 6, and 7. Part of the spread across the later components comes from `mtry` = 1 itself: a tree must sometimes split on a weak component, so every component gets some use and some importance.

(c)

```
yhat = fit_best.predict(Z_test)
err = np.mean(yhat != y_test)
print(f"test misclassification rate = {err:.4f} ({int((yhat != y_test).sum())} of
{len(y_test)})")
cm = pd.crosstab(pd.Series(y_test, name="true digit"), pd.Series(yhat, name="predicted digit"))
print(cm.to_string())
```

```
test misclassification rate = 0.0189 (6 of 317)
predicted digit    1    6    7
true digit
1                102    0    2
6                  0  106    0
7                  0    4  103
```

The test labels are used only in this final step. The selected forest misclassifies **6 of 317** test digits, a **test misclassification rate of 0.019**. Every 6 is classified correctly. Two 1s are predicted as 7 and four 7s are predicted as 6. The test error is close to the OOB error of 0.021, which corresponds to about 7 errors on a set of this size. So the OOB estimate was a good guide to test performance.

Question 3: Planning a random-forest analysis skill

Solution

I would write a skill that provides specific instructions on things that are up to discretion and not part of an AI's existing knowledge base. For example, specific requests on formatting, data preprocessing, model selection criteria, etc. I would not give it instructions on how to do basic things like pass data into an existing library for random forests.

Data Preprocessing (Basic Step): Handle missing values, encode categorical variables, and split data into 80% train and 20% test sets using stratified or random sampling. Baseline Model (Basic Step): Train an initial Random Forest model with default parameters to establish an un-tuned performance benchmark. Hyperparameter Tuning (Basic Step): Conduct 5-fold cross-validation to tune key parameters (n_estimators, max_depth, min_samples_split, max_features). Metric Evaluation (Basic Step): Compute test-set metrics (Accuracy, F1, ROC-AUC for classification; R2, RMSE for regression) alongside Out-of-Bag (OOB) error. Feature Importance (Basic Step): Calculate and rank variable importances using both Gini impurity and permutation methods. Table Formatting (Personal Preference): Output two Markdown tables: one comparing model performance across stages, and one listing the top 10 predictor scores. Plot Styling (Personal Preference): Generate minimalist, slate/navy-colored diagnostic plots, including a horizontal feature importance bar chart and ROC/residual curves. Interpretive Guardrails (Personal Preference): Frame feature importance strictly as predictive association rather than causation, explicitly noting any collinearity effects. Report Layout (Personal Preference): Structure output as an executive summary opening with 3 key takeaway bullets, followed by methodology, performance, and actionable insights. Reproducibility Appendix (Personal Preference): Append a self-contained, fully executable Python script with fixed random seeds (random_state=42) to reproduce all results.